

Coordination for fragmented loops and scopes in a distributed business process

Rania Khalaf^{a,*}, Frank Leymann^b

^a IBM TJ Watson Research Center, 1 Rogers Street, Cambridge, MA 02142, United States

^b Institute of Architecture of Application Systems, University of Stuttgart, Universitätsstraße 38, 70569 Stuttgart, Germany

ARTICLE INFO

Available online 17 September 2011

Keywords:

BPM
Distribution
Recovery
BPEL
WS-Coordination

ABSTRACT

As partner relationships become more dynamic and global boundaries give way to a more agile and dynamic environment, the ability to distribute one's processes in an agile manner becomes increasingly important. Such processes may need to be split not only along their explicit dependencies but also along more complex behavior such as recovery behavior and loops. The resulting process fragments can be distributed and wired together, recreating the execution semantics of the original process model. In earlier work, we presented BPEL fragmentation covering data and explicit control dependencies. We now extend the approach to handle fragmenting loops and scopes. Maintaining the focus on standards and maximizing extensibility of Web service runtimes and standards, the solution defines and uses two new coordination protocols that plug into the WS-Coordination framework. The approach uses the standards as much as is feasible and addresses the remaining required functionality by providing architected extensions. This results in layered approach that maximizes transparency and interoperability. After defining the fragmentation approach for scopes and loops, an implementation is presented that extends the Active Endpoints BPEL engine and a WS-Coordination system. A detailed example is used to illustrate how the protocols are used at runtime to enable the coordinator and the process fragments to recreate the behavior of the original, unsplit process model.

© 2011 Elsevier Ltd. All rights reserved.

1. Introduction

Business process fragmentation consists of taking a process that was modeled as a unit and fragmenting it into several processes owned and executed by a set of partners. There are several drivers for process fragmentation: The business unit that owned the process may need to split apart some business functions to outside partners due to off shoring, spinning off, or outsourcing some of its capabilities, in order to lower its costs and focus on its

core competencies; the process may need to be centrally designed by a set of independent business partners so as to have an agreed-upon end to end definition, but parts of it executed under the technical (and legal) frameworks of each business partner based on their role. It is often important that the fragmentation does not require re-engineering the process and that different partitions of the same process model can be created easily: The business process definitions of a company themselves may constitute an important IP asset, encoding the logic and inner workings that differentiate it from its competitors. In such cases, it is desirable not to modify or reengineer the original processes but instead easily define different partitions on top of it.

Interoperability and transparency are desirable features of a fragmentation solution. The use of standards

* Corresponding author.

E-mail addresses: rkhalaf@us.ibm.com (R. Khalaf), Frank.Leymann@iaas.uni-stuttgart.de (F. Leymann).

provides interoperability, lowering the barrier to entry for a potential partner because he is then more likely to have the skills and tools required to participate. This increases the feasibility of fragmentation. Transparency allows designers and stakeholders of the unsplit business process to maintain their insight and understanding of it once it is fragmented. Transparency can be provided by using the same modeling paradigm for the fragments as was used in the unsplit process and by making it easy to identify the original activities from the unsplit process as well as any process modifications added through fragmentation to transmit data and control between the fragments.

We provide a standards-based approach for fragmentation using the Web services stack of standards, and in particular the Web Service Business Process Execution Language (WS-BPEL, or BPEL) [22] and the WS-Coordination framework [23]. The Web services standards are extensible by design; however, an increase in extensions results in a decrease in interoperability because a partner may not have the appropriate support. We therefore provide a phased approach: Use standards exclusively where possible, identify the situations when the functionality provided by the standards is not sufficient for reproducing the required operational semantics, provide corresponding extensions for the identified cases, and support the extensions in a modular way avoiding forced system replacement.

Throughout this paper, the term ‘unsplit process’ refers to the process model being fragmented, ‘partition’ to the

set-theoretic partition of an unsplit process, ‘split activity’ to a split loop or scope, ‘unsplit’ scope or ‘non-split’ scope to a scope that is not split, and ‘split process’ to the set of process fragments resulting from a partition.

An overview of the approach is shown in Fig. 1: A designer partitions a process (left), defined in the Web Service Business Process Execution Language (BPEL) between different participants such that loops and scopes may be split (thick line). For example, this could be a new-hire on boarding process being split between the HR and Payroll departments based on the activities each department should execute and control. The algorithms then use the partition definition to generate a BPEL process for each participant and a wiring definition describing their connectivity (and a WSDL interface definition, not shown), top right of Fig. 1. These fragments can then be deployed and run in BPEL engines, recreating the operational semantics of the unsplit process by interacting with a coordinator.

In [12,13], we laid the groundwork for fragmenting BPEL processes by enabling splitting data and explicit control dependencies. The approach reproduces BPEL behavior across fragments of a BPEL process by using patterns of BPEL constructs. In summary, an intra-process dependency split by the partition results in an inter-process message exchange(s) between the fragments: a pattern of BPEL sending activities (sending block) is placed in one fragment and a pattern of BPEL receiving

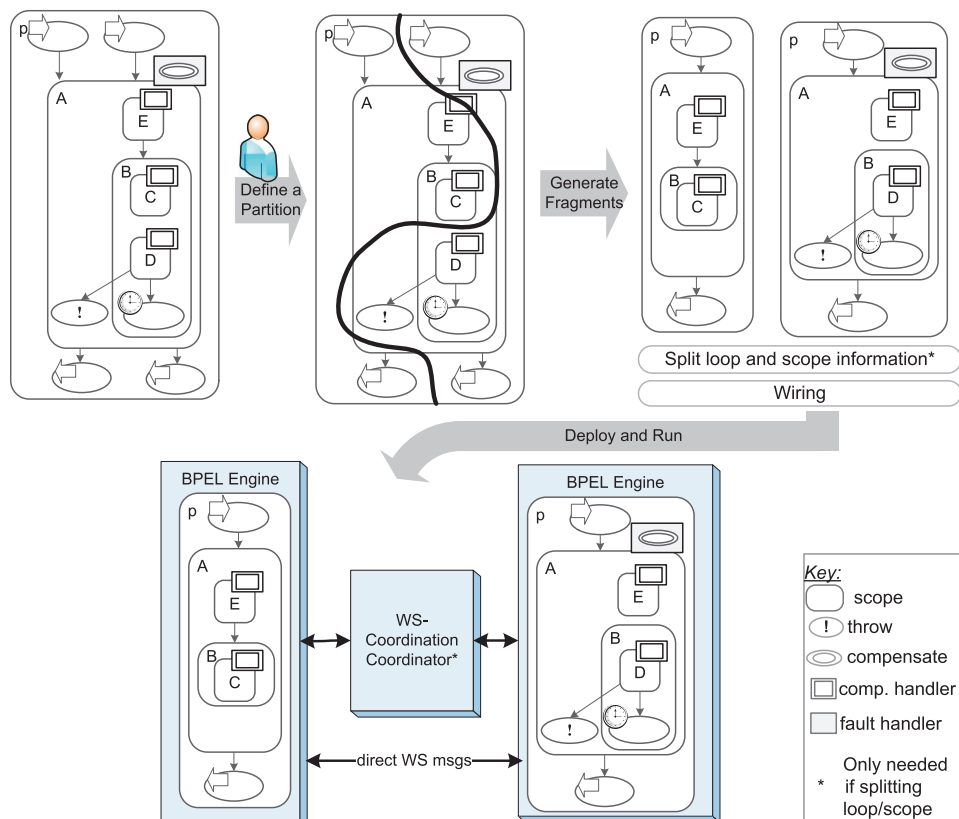


Fig. 1. Approach overview.

activities (receiving block) containing at least one ‘receive’ activity is placed in another fragment. The sending block constructs and sends the needed information while the receiving block receives and handles that information. The patterns are set up such that the result recreates the semantics of the original dependency in the unsplit process. Different types of dependencies result in different patterns of BPEL constructs in the sending and receiving blocks. The algorithms ensure that the fragments are standard BPEL processes.

The focus and contribution of this paper is the fragmentation mechanism for split loops and scopes, from partition specification to execution. We extend the approach in [12,13] to support splitting BPEL loops and scopes.¹ Scopes and loops provide shared behavior to a group of activities: fault handling, compensation handling, and variable scoping for the former and looping for the latter. The activities in a scope or loop serve different purposes and therefore may need to be executed by different organizations or servers. As a result, the enclosing loop or scope is split. We considered also using BPEL patterns for sending and receiving the necessary information for split loops and scopes. However, unlike for splitting explicit dependencies and data, the number of exchanged messages and complexity of the patterns drastically increase with the increase in fragmentation and dependencies for non-trivial loops. For scopes, they become potentially intractable. This is due to fragmenting the corresponding implicit coordination present in these constructs: finding a fault handler across fragments, propagating faults, synchronizing start/end of fragmented loops, scopes and handlers. A single fragment may also not have enough information, such as determining and enacting default compensation order. Thus, we find splitting scopes and loops to be a clear point where language and runtime extensions provide a better solution.

The extensions, compliant with the extensibility of the standards, created for this solution consist of three new BPEL attributes for scopes and loops and two new coordination protocols that plug into the WS-Coordination framework. The overall approach is layered: data and explicit control dependencies are split according to [12,13] using plain BPEL; implicit dependencies due to split loops and scopes are split using coordination as described in this paper if the process has at least one fault handler and/or contains at least one split activity. The process in Fig. 1 does not have the former, so no messages need to be exchanged directly between its fragments.

Section 2 presents relevant aspects of BPEL and WS-Coordination. The structure then follows the steps of the approach: Section 3 presents split scopes and loops and the corresponding BPEL language extensions. Section 4 presents the use of coordination for this problem, the split process information needed by the coordinator, and the new coordination protocols. Section 5 describes the implementation. We then present a detailed example, discuss related work and conclude.

2. Background: BPEL and WS-Coordination

BPEL scopes and loops are structured activities, thus having activities nested within them. Structured activities are strictly nested and may be the source or target of explicit, conditional control links. An activity with incoming links has a ‘join condition’ that by default is the disjunction of the values of the incoming link conditions. If it evaluates to false, the activity throws a join failure fault that may potentially be suppressed. Suppressing it results in skipping the activity and setting its outbound links to false. A BPEL loop is similar to a while loop in programming languages. Explicit control links must not cross the loop boundary, but they may cross a scope’s boundary. A BPEL scope, among other things, defines fault and compensation handlers. A fault or compensation handler may initiate compensation on one or more of its immediate child scopes: explicitly naming a target scope using the ‘compensateScope’ activity or all immediate children in default order using the ‘compensate’ activity. The default compensation handler consists of a ‘compensate’ activity.

If a fault is thrown by an activity nested in a BPEL scope, it terminates all its nested activities, looks for a local fault handler, and runs the handler if it finds one. Once the fault handler completes, navigation continues at the scope boundary. If a handler is not found, the default fault handler runs: terminate all nested running activities, run ‘compensate’, then rethrow the fault to the parent scope.

Termination of a scope results in triggering the default termination handler, which consists of a call to ‘compensate’. All links leaving a terminated activity are fired negatively. Notice that when fault handling, faults propagate up the scope hierarchy, with the order of aborting siblings being irrelevant. On the other hand, default compensation occurs one level of scope nesting at a time and in an order that reverses control dependencies between peer scopes. In-depth scope behavior is described in [14].

WS-Coordination is a standard providing a pluggable framework for coordinating agreement between a set of services working together towards a joint outcome. The standard defines an Activation Service and a Registration Service. The optional Activation Service is used by the initiator of the coordination to create a unique ‘coordination context’ that identifies the particular instance of the distributed action being coordinated. This context is then exchanged in the header of the application messages that are part of the distributed action. The Registration Service is used by services wishing to participate in the distributed action. Usually, a service registers upon receiving a message with a coordination context. The protocol used to coordinate the distributed action is pluggable and is identified in the exchanged messages. Upon registering, each service must refer to a corresponding protocol handler that is in charge of exchanging messages according to the chosen protocol. Defining a new coordination protocol consists of defining participant and coordinator protocol services and handlers, the protocol message exchange order, and the required underlying behavior of the coordinator and the participants in relation to the exchange of these messages. One may also extend the activation and registration messages.

¹ Description of the protocols first shown in our technical report at http://elib.uni-stuttgart.de/opus/volltexte/2007/3050/pdf/TR_2007_01.pdf.

3. Splitting loops and scopes

A designer wishing to split a process amongst a set of participants defines a split process wherein he assigns different parts of the process to the different participants. More precisely, the designer defines a split process, as in [12], by specifying a partition of the set A of all primitive activities of the process. Consider P_n , the set of participant names. Every participant p in the set of participants P_a consists of a participant name in P_n and one or more activities such that $P_a \subseteq (P_n \times \wp(A))$. The following restrictions apply to the partition:

- A participant must have at least one activity.
- No two participants share the same primitive activity.
- Every primitive activity is assigned to a participant.

The splitting algorithm [12] uses this information to create one BPEL process and one WSDL for each participant, in addition to wiring information that is used to connect the fragments together at runtime.

The partition definition is used to determine if a scope or loop is split: If all activities in the body of a loop or scope or in the handlers of a scope are not assigned to the same participant, then that loop or scope is split. For example, scope E in Fig. 1 is split but scope C is not. A handler is split if all the activities in the handler body are not assigned to the same participant. For loops, the designer also has to designate one participant as responsible for the loop condition. Let L_n be the set of split loop names. Thus, we define a map L_c that associates a participant with every split loop name: $L_c : L_n \rightarrow P_n$. The splitting algorithm then places a fragment of the scope or loop in each process fragment that has at least one of the activities of the scope or loop assigned to it. The algorithm denotes a loop or scope as split in the resulting process fragments using the following language extension attributes on the corresponding XML elements:

- On $\langle \text{process} \rangle$: 'belongs-to' whose value is the QName of the unsplit process.
- On $\langle \text{scope} \rangle$ and $\langle \text{while} \rangle$: 'fragmented' whose value is 'yes' or 'no' (default 'no'). When set to 'yes' it identifies the activity as split. It must be set to 'yes' in all fragments of the split activity.
- On $\langle \text{while} \rangle$: 'is-responsible' whose value is 'yes' or 'no' (default 'no'). A loop having this attribute set to 'yes' is the fragment responsible for evaluating the loop condition. Such a loop must be split and exactly one of its fragment must have the value of 'is-responsible' be 'yes'.

The restrictions for this solution are: (a) all split activities are uniquely named in the unsplit process model, (b) compensation is a recovery mechanism and thus must not fail, and (c) scope fragmentation is focused on fault and compensation handling thus not covering variable scoping and event handling. These are in addition to restrictions carried over from [12], including: (d) the fragments share a common correlation set serving as the split process instance identifier, (e) a 'receive' and its corresponding 'reply' are

disallowed from being placed in different participants, (f) the split activities covered are 'loop', 'scope', and 'flow' with no incoming/outgoing links. Such a 'flow' (not shown in Fig. 1) is needed because BPEL syntax requires the body of a loop, scope or handler have exactly one (structured or primitive) activity.

Supporting the rest of the BPEL structured activities is possible without loss of generality and in the case of 'pick' with an update to the partition definition. As this is not in the scope of this paper, we briefly mention that an enabler for doing so is the ability to flatten these activities as shown in [10], using a start activity, an end activity, and a pattern of activities between them connected with links to reconstruct the control imposed by the structured activity. The 'pick' activity, representing external choice, requires us to restrict users from splitting the choice itself while allowing splitting its branches.

As one can see from the partition definition, the fragmentation supports splitting the fault and compensation handlers of a scope in addition to splitting the body of the scope. A fault handler may be in a different fragment than the one that the fault originated in. Splitting a scope requires being able to synchronize the start and end of the fragments. Splitting a scope with a fault handler requires the ability to abort execution in all fragments of that scope, find a matching fault handler, and synchronize the recovery behavior up to and including the scope with the handler. Splitting a scope with compensation behavior requires not only synchronizing the execution of a split compensation handler, but also being able to determine and enforce the scope's default compensation order. The compensation order cannot be handled purely locally because child scopes may be in several fragments. Thus, neither fragment alone has enough information regarding what order to compensate its nested scopes in, nor whether any interleaving is necessary with compensation of scopes in the other fragments. Enabling the corresponding behavior requires the coordinator to know the scope hierarchy, which fault handlers are in what fragments, and the default compensation order calculated from the unsplit process. We identify the information needed and encode it in two data structures: the Relationship Tree and the Default Compensation Order (DCO graphs). These data structures are detailed in Section 4.2.

3.1. The rubber band effect

Split processes exhibit a property we call the *rubber band effect*: BPEL scopes and loops are strictly nested; it follows therefore that this nesting and resulting nesting relationships must be maintained upon splitting. Scope E in Fig. 1 is split between two participants, so its ancestors A and the process itself also have fragments in those participants. A corollary is that if x is a split scope or loop, all ancestor scopes and loops of x (including the process), must also be split. If this property is violated, the split process could end up in an inconsistent state.

3.2. Links from and to split activities

In the case that a split loop or scope is itself the source or target of a control link, the algorithms must determine

on which fragment of the loop/scope to place the link. That is, for each link, $l=(a,b)$ from activity a to activity b and where either a , b or both are split: if a is split, we need to choose one fragment, $f_{source,l}$, of a as the source of l in the split process; if b is split, we need to choose one fragment, $f_{target,l}$, of b as the target of l in the split process. Note that the link placement determines which fragment of a link's source/target is used and does not change the source or target.

Regarding notation, we use $\pi_i(t)$ to denote the projection function (projection to the i -th component of a tuple t) and represent a link by a source activity and target activity tuple.

For an activity that is the target of one or more links, a fragmentation approach has to determine where to evaluate the join condition. Fig. 2 shows a split scope on top with a join condition jc and two incoming links from two activities, a and b . On the bottom, it shows three ways of splitting the process and its impact on the join condition. Fig. 2, case 2 shows that the choice of target fragments when splitting an activity is affected by the condition: if it is not considered, then it is no longer clear who is responsible for evaluating it or what its value is. Recall that the join condition is a condition of an activity on the status of all incoming links that is evaluated once all the links have fired. A link's status is the value of evaluating the link's transition condition when the link fires. We aim to keep the business logic in the processes, having the coordinator focused on navigational not business behavior. The join condition is part of the process's business logic, so we do not wish to place its evaluation in the coordinator. Therefore, we require that it be evaluated at one fragment (Fig. 2, cases 1 and 3). In the algorithm described in this section, we keep the join condition intact by choosing one fragment to be the target of all the control links having the split activity as their target. We can relax this restriction if the join condition is (or can be restructured to be) the conjunction of the local join condition of each fragment. Conjunction is chosen because the coordinator will already be synchronizing all the fragments so that they may start together. For example, the restriction is thus relaxed for the links into Scope A in Fig. 1 so as to focus the paper on the coordination aspects. Otherwise, one of the control links to A would be split and they would both have one fragment as their target.

Consider a function $getTarget(a)$ that takes a split activity a as input and returns the fragment that has the lowest cost if one is chosen to be the target of all a 's

incoming links. Consider another function $getSourceAndCost(l)$ that takes a link l as input and returns a tuple containing the fragment having the lowest cost if it is chosen to be the source of l and the cost of choosing it. If more than one fragment has the lowest cost, then the choice between them is arbitrary.

For every split activity a that is the target of one or more links, we choose the fragment with the lowest cost of being the target of all these links. Thus, $f_{target,l} = getTarget(a)$. Having covered links whose target is a split activity, we then choose the source fragment of each link whose source is a split activity. If the link's target is not split, $f_{source,l} = \pi_1(getSourceAndCost(l))$. If the link's target is split, we set the chosen target fragment as the link's target and choose $f_{source,l} = \pi_1(getSourceAndCost((l.source, f_{target,l})))$.

The rest of this section explains what affects the cost, defines the $getTarget$ and $getSourceAndCost$ functions, and shows some examples.

Different choices of the fragments for link placement for split loops and scopes result in potentially different dependencies getting split. We would like to make the choice that causes the least dependencies to be split because each split dependency requires messages to be exchanged between the fragments. Thus, we aim to choose the fragments so as to reduce (and ideally to minimize) the number of inter-fragment messages added due to assigning the link source/target fragments. In this case, such messages would be caused by (i) splitting the link itself, (ii) splitting any data dependencies for the link's transition condition. The creation and exchange of these messages is done using sending and receiving blocks of BPEL activities. One could choose arbitrarily between all fragments and then use the algorithms in [12] to handle any resulting split data or explicit control dependency. However, we can do better by choosing the fragment(s) based on an estimate of how many messages would need to be exchanged. We do so by associating a cost that estimates the number of messages added for each choice of source/target fragment.

For (i), the link itself is split if the source and target are in different participants, resulting in at most one message that propagates link status.² Consider a function $splitControlCost(a, b)$ that takes two activities or fragments as input and returns 1 if they are in the same participant and 0 otherwise. We will use this to associate a cost for choosing the source and/or target fragments for a link:

$$splitControlCost(a,b) = \begin{cases} 1 & !inSameParticipant(a,b) \\ 0 & \text{otherwise} \end{cases}$$

For (ii) splitting one or more data dependencies for the link's transition condition occurs if the link's source is a split activity and the fragment chosen as the source needs data from other fragments in order to evaluate the transition condition. Note that the transition condition is evaluated in the fragment containing the source of the

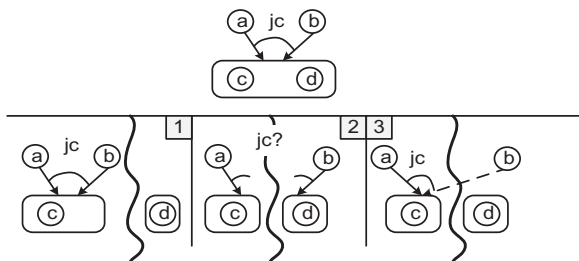


Fig. 2. The effect of splitting and target fragment choice on the join condition [10].

² If one splits a data and a control dependency whose sources and targets are the same, then one message is used to propagate both the data and the control [13].

link. We want to associate a cost, due to splitting data dependencies, for choosing a fragment f of the split activity as the source of link l . We use the term ‘writers’ to refer to the activities that write data required by a reader (activity or transition condition) whose writes may reach the reader. The writers are found by performing data analysis on the process [19,15]. Splitting BPEL data dependencies is non-trivial and detailed in [12], which generates the activities to exchange these messages and thus can provide the exact number. Here, we use an approximation: one message for every writer that is in a different fragment than the reader. Let $tcWritersExternal(l, f)$ for a link l and fragment f be a function that returns the set of writers of the transition condition of link l that are not in the same participant as f . Thus, we associate a split data dependency cost for choosing f as the source of l to be the number of writers in $tcWritersExternal(l, f)$. This is an over-approximation because [12] performs some optimizations locally before sending data to a fragment. One could further improve this split data cost function by taking into account the details of [12] and a heuristic on the expected sizes of the messages. It is important to note that the default transition condition in BPEL does not read any data.

$$splitDataCost(l, f) = |tcWritersExternal(l, f)|$$

We are now ready to define $getSourceAndCost(l)$, which is called if the link's source is a split activity and shown in Fig. 3: (lines 4–5) For each fragment of the split activity that is the source of the link, set the cost to be the sum of the split data dependency and split control dependency costs. (lines 6–9) Return the fragment for which this cost is lowest along with the corresponding cost.

The $getTarget(a)$ function, called for a split activity that is the target of links, is shown in Fig. 4. It does the following: (lines 4–5) For each fragment f of the split activity, calculate the cost of making that fragment the

target of all the activity's incoming links and thus responsible for the join condition. This cost is the sum of two values corresponding to two subsets of a 's incoming links: L_u^+ , the subset whose sources are unsplit and L_s^+ , the subset whose sources are split. The cost for each link in the former is the split control cost. For each link in the latter, the cost also depends on the choice of source fragment. To locate the best source fragment, we call $getSourceAndCost$ using f as the target.

To illustrate, consider an example regarding outgoing links such as the one from the scope to d in Fig. 5. The figure shows an unsplit scope on top and different partitions and link placement on the bottom.

The question here is which scope fragment should be the source of the link (e, d) if scope e is split (cases 1 and 2 of Fig. 5). This is found using $getSourceAndCost((e, d))$.

1. In case 1, assume that the transition condition does not read any data written by the fragment on the left. The cost of choosing the left fragment is 1 because that would separate the source and target while that of choosing the right fragment is 0. Thus, the fragment on the right (as shown) will be the source of the link to d .
2. In case 2, assume the transition condition of the link

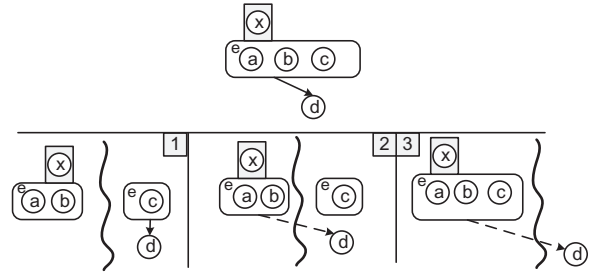


Fig. 5. Options for outgoing links.

```

1. function getSourceAndCost(l)
2.   sourceFragment = ∅
3.   lowestCost = 0
4.   for each fragment f of l.source
5.     cost = splitDataCost(l, f) + splitControlCost(f, l.target)
6.     if sourceFragment = ∅ || cost < lowestCost
7.       sourceFragment = {f}
8.       lowestCost = cost
9.   return (sourceFragment, lowestCost)

```

Fig. 3. Pseudo-code for $getSourceAndCost(l)$.

```

1. function getTarget(a)
2.   targetFragment = ∅
3.   lowestCost = 0
4.   for each fragment f of a
5.      $jcCost = \sum_{i=0}^{|L_u^+|} splitControlCost(l_i.source, f) + \sum_{i=0}^{|L_s^+|} \pi_2(getSourceAndCost(l_i.source, f))$ 
6.     if targetFragment = ∅ ||  $jcCost < lowestCost$ 
7.       targetFragment = {f}
8.       lowestCost =  $jcCost$ 
9.   return targetFragment

```

Fig. 4. Pseudo-code for $getTarget(a)$.

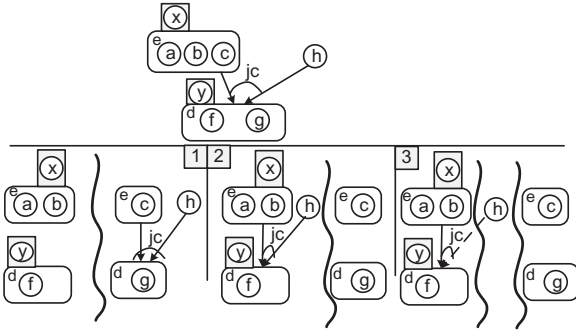


Fig. 6. An example of choosing source and target fragments for links.

reads data from activities *a* and *c*. Now, the cost for choosing the fragment on the left is 1 due to the splitControlCost of splitting the control link. The splitDataCost is 0 because all writers of the transition condition are in the same participant as the source fragment. The cost for choosing the fragment on the right is 2 because two writers are in another participant.

3. Case 3 shows that if the scope is not split, link (*e*, *d*) keeps the scope as its source and is handled by the splitting of explicit links in [12].

Next, consider a more complex example where both source and target are split scopes as shown in Fig. 6. Assume the transition conditions are the default, in which case splitDataCost is always 0. Applying the algorithm above yields the source and target choices shown in the bottom row of the figure. In case 3, the left-most fragment of *d* is shown as the target of both (*e*, *d*) and (*h*, *d*) and the source of (*e*, *d*). In fact, there is a tie between the two fragments of *d*. Either choice is valid provided that the same process fragment is used for all three links. In other words, let f_d be the returned fragment by $getTarget(d)$. Thus, f_d could be either fragment of *d* because the cost is equal. Then, $getSourceAndCost(f_d)$ also returns f_d as the source of (*e*, *d*).

The coordination protocols described in the next section will ensure that a link whose source is a fragmented activity will not fire unless all fragments have completed and that an activity whose target is a fragmented activity will not evaluate its join condition until all incoming links have fired.

4. Using coordination

Split loop and scope execution can be cast as a coordination problem: each fragment performs part of the distributed action corresponding to the behavior of the unsplit activity. For example, the split loop protocol is about agreeing whether or not to iterate again. Our protocols, by design, do not replace existing BPEL engine behavior; they complement and leverage it to enable the engines to collectively recreate the operational semantics of the unsplit process. We assume that the choice of where the coordinator will run is determined by the participants and the unsplit process owner. It could be

run by a trusted third party, the entity responsible for the unsplit process, or one of the participants.

At first glance, it seems that the WS-BusinessActivity (WS-BA) [23] protocol could be used to coordinate scope fragments; however, this is not the case for several reasons including the fact that the fragments are peers whereas WS-BA was created to perform the coordination between a parent scope and its child scopes instead of fragments of one scope. The coordination for split loops and scopes differs from the traditional use of WS-Coordination such as WS-BA and WS-AtomicTransaction. The fragments of loops and scopes are known at design time and their location is known at deployment time. The fragments are peers and it is not known ahead of time which will start the protocol: i.e.: either process fragment in Fig. 1 can receive the first message to create an instance; once an instance is created, either fragment of scope *A* can be reached first in the navigation. Our protocols are driven by lifecycle events and not by application messages.

We do not use the optional Activation Service for two reasons. First, a protocol instance start is automatically detected when the first fragment of a split activity is reached in the navigation. Second, each participant has enough information to enable the coordinator to determine which protocol instance it belongs to: the fragments, their locations, the unsplit activity they are part of, and the instance of the process they are part of are known by the time the first coordination message is sent. There is therefore no need in our solution to flow the coordination context. The required information, defined in Section 4.2, consists of design-time information derived from the unsplit process, encoded in the relationship tree and Default Compensation Order (DCO) graphs, deployment time information in the wiring model, and runtime information consisting of the common correlation set value.

4.1. Deployment and registration

To deploy fragments containing split loops or scopes, we extend the wiring model in [12]. The model provides connectors between the fragments themselves as well as between the fragments and external services invoked by the unsplit process. We now add a ‘coordination’ element containing: (1) the address of the registration service of the coordinator so that the first process fragment to start an instance can register; (2) the endpoint of a ‘starter service’ for each process fragment.

Protocol registration requires providing a URL to serve as a protocol identifier; therefore we create one for each of the protocols. We also extend the registration message to include additional information needed to uniquely identify the instance of coordinated work and the participant registering for it.

The extensions are new sub-elements of the registration message’s ‘wscoord:Register’ element:

- $\langle \text{ProcessName} \rangle$, the name of the split process the activity is in;
- $\langle \text{LoopName} \rangle$ or $\langle \text{ScopeName} \rangle$, the name of the split activity;

- $\langle \text{ParticipantName} \rangle$, the name of the participant the registration is for, used to determine which fragment is registering and match that to the information in its relationship tree;
- $\langle \text{CorrelationSetValue} \rangle$, an identifier of the instance of the coordinated work. Its value is the value of the common correlation set, in effect identifying the instance of the split process;
- $\langle \text{Counter} \rangle$, a counter that further identifies which instance of a scope or loop is being coordinated, needed to distinguish multiple iterations of the split activity in the same process instance.

Traditionally, the identifier of a specific instance of coordinated work (i.e. transaction id) is flowed in the context after calling the activation service. However, in this case the instance identifier is already known by all fragments and can be placed in the registration message thereby making the activation service superfluous.

4.2. Encoding common information

The a priori information required by the coordinator about the unsplit process model is encoded in a scope and loop relationship tree and DCO graphs. They may be created at any time before the first process fragment creates an instance. They may be passed to the coordinator any time before a process fragment begins, such as at deployment time or in the registration message. Details of the formalism of these data structures and their construction are in [10, Chapter 5].

4.2.1. Scope and loop relationship tree

The relationship tree, $RT = (N_{rt}, E_{rt})$, encodes information about the nesting relationships of split activities, their handlers, and their fragments. It is created from the unsplit process model and partition definition. An example is shown in Fig. 7.

The set of nodes N_{rt} is divided into four pair-wise disjoint sets:

- Split scope node set S_{rt} : Each scope node s_{rt} is a tuple of a scope name, a set M_s of faults the scope has a handler for, a Boolean stating whether the scope is in a handler

of its immediately enclosing scope, and a Boolean whether it is in the compensation handler of its immediately enclosing scope. Thus, $s_{rt} = (\text{name}, M_s, \text{in_fault_handler}, \text{in_comp_handler})$.

- Unsplit scope relevant node set S_{ns} : Includes scopes that are not split but need to be compensated by their ancestor split scopes. These are unsplit scopes where: their parent scope is split and they contain an explicit compensation handler either directly themselves or in any scope nested within them. Each such node s_{ns} is a tuple consisting of the participant name it is in, the scope name, and a Boolean whether it has an explicit compensation handler. Thus, $s_{ns} = (\text{p_name}, \text{name}, \text{has_compensation})$.
- Split loop node set L_{rt} : Each loop node, l_{rt} , is a tuple containing the name of the loop. Thus, $l_{rt} = (\text{name})$

- Fragment node set F_{rt} : A fragment node represents one fragment of a split activity and is a leaf node in the tree. They are of two kinds, such that

$$F_{rt} = F_{rt_s} \cup F_{rt_l}$$

- Loop fragment nodes, F_{rt_l} : Each loop fragment node is a tuple containing the name of the participant it is in, the name of the loop it is a fragment of, and a Boolean whether it is responsible for the loop condition.
- Scope fragment nodes, F_{rt_s} : Each scope fragment node is a tuple containing the name of the participant it is in, the set O of fault names this particular fragment has a handler for, the name of the scope it is a fragment of, and a Boolean whether the fragment has an explicitly defined compensation handler.

The set of edges of consists of three pair-wise disjoint sets, $E_{rt} = A_{rt} \cup B_{rt} \cup C_{rt}$:

- The set of fragment-activity edges, A_{rt} , connecting a fragment node to the loop or scope node it is a fragment:

$$A_{rt} \subseteq (F_{rt_s} \times S_{rt}) \cup (F_{rt_l} \times L_{rt})$$

- The set of loop-scope edges, B_{rt} , connecting a loop to its parent scope:

$$B_{rt} \subseteq L_{rt} \times S_{rt}$$

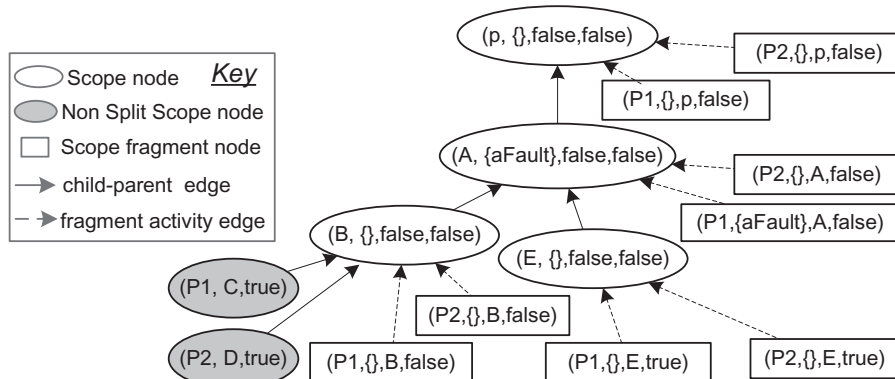


Fig. 7. The relationship tree for the example in Fig. 1.

- The set of child–parent edges, C_{rt} , connecting a scope to its immediately enclosing scope. Each scope node has exactly one such edge except for the root which has none. A scope in a fault handler of another scope is considered nested in the latter:

$$C_{rt} \subseteq (S_{ns} \cup S_{rt}) \times S_{rt}$$

4.2.1.1. Searching for a fault handler. In this section, we present the algorithm that finds the fault handling scope fh_{rt} for a particular fault by using the relationship tree and information about where the fault originated. This information will be sent to the coordinator in a fault related message by the participant in which the fault occurs.

Consider s_r to denote the root scope. First, we define a function called $parent_{rt}$ that retrieves the immediate parent of a given scope using the scope tree. The root scope has no parent, so the domain of $parent_{rt}$ is $S_{rt} \cup S_{ns} - \{s_r\}$. In what follows, the symbol $\perp$ represents ‘not found’. Recall from Section 3 that $\pi_i(t)$ is the projection function:

$$parent_{rt}(s) := \pi_2(c) \quad \text{where } c \in C_{rt} : c = (s, s')$$

To check whether a particular scope has a handler for the fault, we define a function $m_{rt}(n, s)$, taking the name of a fault and a split scope node as input:

$$m_{rt}(n, s) = \begin{cases} \text{true} & n \in \pi_2(s) \\ \text{false} & \text{otherwise} \end{cases}$$

Recall that one must skip the parent scope when a fault is crossing a fault handler boundary and that we are using ‘ $\perp$ ’ to represent ‘not found’. The coordinator uses a function $g_{rt}(s)$ that takes a scope node from the relationship tree as input and returns the next scope to search by using the relationship tree. Recall that for a scope node s , $\pi_3(s)$ is a Boolean stating whether or not the scope is in the fault handler of its immediately enclosing scope. This determines whether to skip the scope or not. Therefore, we define $g_{rt}(s)$ as follows:

$$g_{rt}(s) = \begin{cases} parent_{rt}(s) & \neg \pi_3(s) \\ g_{rt}(parent_{rt}(s)) & \pi_3(s) \wedge s \neq s_r \\ \perp & \text{otherwise} \end{cases}$$

The scope where the search starts is either the one where the fault occurred or one of its ancestors in case it originated from an unsplit scope. Let the scope node in the relationship tree corresponding to the split scope that receives the fault-related message be s'_{rt_0} . Let s_{rt_0} be the BPEL scope where the search is to start, n the fault name in the message from the participant, and h a Boolean value that is true if the fault-related message was *faultedInHandler* and false if it was *faulted*. These messages will be explained in Section 4.3. Therefore:

$$s_{rt_0} := \begin{cases} s'_{rt_0} & \neg h \\ g_{rt}(s'_{rt_0}) & h \wedge s'_{rt_0} \neq s_r \\ \perp & \text{otherwise} \end{cases}$$

To retrieve the scope with the matching handler, we define a function $y_{rt}(n, s)$ that takes the name of the fault and the scope where the search must begin and returns

either the fault handling scope or $\perp$ if none is found:

$$y_{rt}(n, s) := \begin{cases} s & m_{rt}(n, s) \wedge s \neq \perp \\ y_{rt}(n, g_{rt}(s)) & \neg m_{rt}(n, s) \wedge s \notin \{s_r, \perp\} \\ \perp & \text{otherwise} \end{cases}$$

Finally, the scope that can handle the fault is found as follows: The fault notification message sent to the coordinator provides both h and n and is addressed to the protocol of s'_{rt_0} . The coordinator then uses h , n and s'_{rt_0} and the relationship tree to find s_{rt_0} . Then, $fh_{rt} := y_{rt}(n, s_{rt_0})$ returns either the fault handling scope if one is found or $\perp$ otherwise.

4.2.2. Default compensation order graphs

A DCO graph, $G_D = (N_D, E_D)$, is a Directed Acyclic Graph that encodes the default compensation order of a scope. One DCO is created for each scope that has at least one explicit compensation handler nested within it and zero or more ‘loopDCOS’ for compensation relevant loops. Coordination only requires DCO graphs for split scopes, although the graphs are not dependent on a particular partition.

The set of nodes N_D of a DCO graph of a scope s contains loop and scope nodes. A loop node is a member of N_D if the loop is (1) an immediate child of s , (2) not nested in another loop in s and (3) contains at least one scope having an explicit compensation handler nested at any level in s . A scope node is a member of N_D if it is (1) an immediate child scope of s , (2) not in handlers of s , (3) not nested in a loop that is in the DCO node set, and (4) either it or any of its nested scopes has an explicit compensation handler.

For each loop node in the DCO graph of a scope, there is an associated loopDCO graph $G_l = (N_l, E_l)$. The nodes in N_l have similar characteristics to those in N_D except using the corresponding loop instead of the scope s to determine the relevant loop and scope.

The edges E_D of a DCO or loopDCO represent the reversal of the control path between the nodes of the DCO or loopDCO: an edge is present in E_D (or E_l) between two nodes x and y in N_D (or N_l) if there exists a control path in the process from y or any of its nested activities to x or any of its nested activities such that the path does not contain any other node in N_D (or N_l).

Running compensation in default order occurs by navigating the DCO as a BPEL process with all join and transition conditions set to true. Book-keeping for instances can be encoded in ‘ScopeAndLoopQ’ structures in [10,14]. Upon navigating to a node, it is run and navigation continues by following its outbound edges. Running a scope node consists of a no-op if it is not completed, running its compensation handler if it has an explicit one, or running default compensation by navigating its DCO. Running a loop node consists of navigating the corresponding loopDCO, in the same way, as many times as the loop had run for that instance of the scope/loop in which the loop node is nested.

4.3. The coordination protocols for split loops and scopes

A protocol is provided as a state machine encoding the messages, the order in which they are exchanged and the description of the corresponding behavior required of

the coordinator and the participants. Each state corresponds to a point where a message is sent by either the coordinator (solid line) or the participant (dashed line). The coordinator executes the protocol with each instance of a fragment of a split loop or scope. The coordinator uses the relationship tree to determine when all fragments have registered, search for handlers, and relate endpoints to participant names. The state machine for the split scope protocol is provided in Fig. 9.

We explain the protocols using the subsets identified in the left margins: common and loop for the loop protocol and adding fault handling and compensation handling to common for the scope protocol. After explaining the protocols, we will show in Section 6 how the protocols are used to enact the split process in Fig. 1.

4.3.1. Common split activity subset

Consider the protocol subset common to all split activities, on the top row of Figs. 8 and 9. It synchronizes the beginning of a split activity: When a fragment is reached in the navigation, it either starts and sends *starting* or it is skipped due to a 'joinFailure' fault it needs to suppress and sends *joinFailure*. Whether it suppresses a joinFailure depends on the activity's suppressJoinFailure attribute. The fragment then waits until the coordinator has heard from *all* fragments and sends either *completeAfterFault* if at least one fragment sent *joinFailure* or *start* otherwise. In the former case, the protocol ends in the 'dead path' state, otherwise it goes to 'active'. The protocol

also synchronizes the end of a split activity: Once a fragment is ready to complete, it sends *completing* and then waits until the coordinator has received *completing* from all the fragments and sends it the *complete* message. The protocol then ends in the 'completed' state.

4.3.2. The loop protocol

The loop protocol is shown in Fig. 8. It adds two transitions to the common split activity subset, both for messages sent by the coordinator: *continueLoop* from 'completing' to 'active' and *complete* from 'start' to 'completed'. The fragment responsible for the loop condition sends the value of the condition in the *start* and *complete* messages to the coordinator. Once all fragments start, the coordinator sends *start* if the condition is true and *complete* otherwise. Once all fragments complete, the coordinator sends *continueLoop* if the condition is true or *complete* otherwise.

4.3.3. The fault handling subset of the scope protocol

Data provided with a fault is sent to the coordinator, which then sends it to all fragments that have a matching handler. A fault handler at a fragment that has a 'fault-Variable' attribute will then save this message in this variable. The scope protocol is shown in Fig. 9.

Messages for fault handling are added from the 'active' state. Two special cases apply if the split scope is the process itself: (1) Upon receiving *start* from the first process fragment to begin, the coordinator sends a 'startInstance' message containing the value of the correlation set to the starter

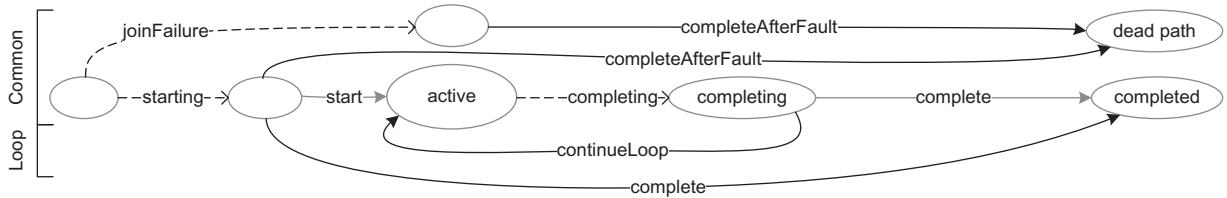


Fig. 8. The split loop protocol.

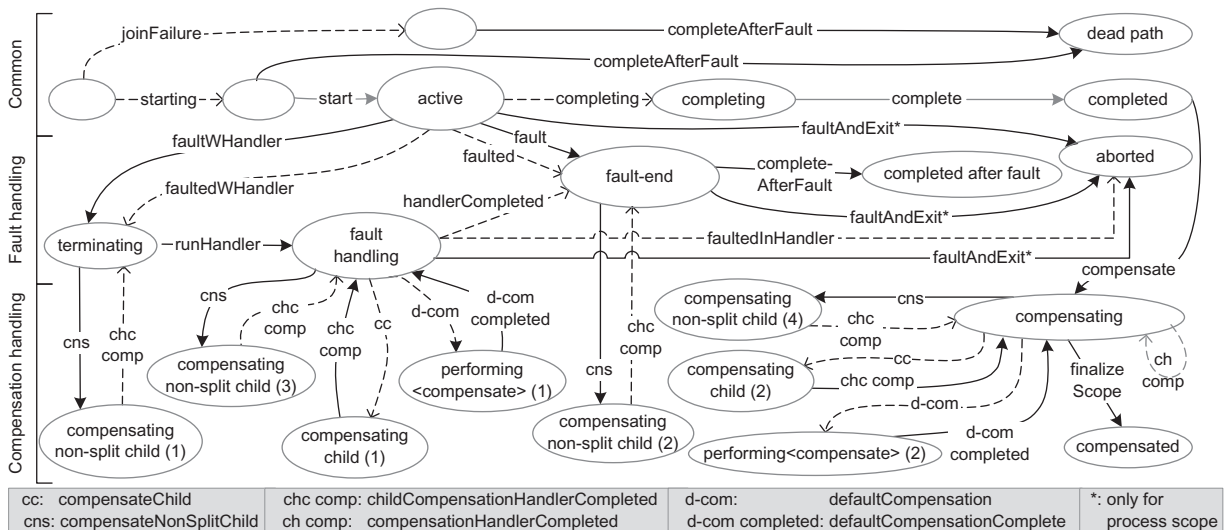


Fig. 9. The split scope protocol.

service of each of the other process fragments. As each fragment creates its process instance, it will send a registration message to the coordinator and the protocol proceeds as for any non-process scope. (2) A process fragment may receive a *faultAndExit* in the case that a fault is encountered in the split process for which no matching handler is found. The rest of the messages apply to all fragmented scopes.

If the fragment of a scope s_m encounters a fault, it sends either *faulted* if it has no matching fault handler in its fragment or *faultedWHandler* otherwise. Either causes the coordinator to search the tree for the scope s_f in whose protocol it will continue: s_f is the scope having a matching fault handler or the process if no handler is found. In our example, the throw activity causes scope A's fragment to send *faultedWHandler* and the coordinator finds that $s_f = A$. We now consider only the fault subset, adding messages and behavior related to potential compensation for this case in the compensation handling subset below. The coordinator first aborts the coordinator-side of scope and loop protocols nested in s_f (except for scopes in the 'completed' state). If there is no handler, it sends *faultAndExit* to the fragments of $s_f = \text{process}$. If a handler is found, regardless of whether $s_f = s_m$ or one of its ancestors, the coordinator sends *faultWHandler* to the fragments of s_f that have the handler and *fault* to the rest. The *fault*, *faultWHandler*, *faultAndExit* messages throw the fault in the scope at each process fragment, causing local engine behavior to abort nested scopes and loops. Both the participant and the coordinator sides of nested split activity protocols are therefore stopped for all fragments and no separate 'abort' message is needed. We require that messages arriving (i.e. due to a race) at the coordinator side for an aborted protocol are ignored.

The protocol synchronizes the beginning and end of split fault handlers. Therefore, a fragment having a matching handler waits for the coordinator to send *runHandler* before starting it. The handler in a fragment then either completes (signaled by sending *handlerCompleted*) or encounters a fault. In the former case, the coordinator waits for all handler fragments to complete and sends *completeAfterFault*. In the latter case, the participant sends *faultedInHandler*, placing the protocol in the 'aborted' state and causing the coordinator to search the tree for and notify the scope containing a matching fault handler.

4.3.4. The compensation handling subset of the scope protocol

Fragmented compensation handling requires coordinating the start and end of fragmented handlers as well as determining and enforcing compensation order. The latter is not possible locally because child scopes may be in multiple fragments. BPEL engines extended to support split activities are not allowed to calculate default compensation order of a split scope or any of its nested children. The coordinator will use the DCOs and relationship tree to enforce and drive compensation in the correct order. Thus, the extended BPEL engines may only start an explicit compensation handler of a split scope and can only do so upon receiving a request from the coordinator. For unsplit scopes in the relationship tree (S_{ns}), the engine

can only start their explicit compensation handler if the coordinator requests it. Since there is no protocol for unsplit scopes, this request arrives via the protocol of an ancestor split scope. Both the engine and the coordinator may trigger the default compensation handler of scopes in S_{ns} . The engine will do so if default compensation is required by the scope's termination behavior.

Consider the part of the protocol for compensating a split scope, e.g. scope E in the example. It is covered by the states reachable from 'completed' because a BPEL scope may only be compensated if it has completed. If a coordinator needs to compensate E, it sends *compensate* to all fragments of E. If the fragment has a compensation handler, it will run the handler and send *compensationHandlerCompleted* upon handler completion. Once all the handler's fragments have sent this, the coordinator sends *finalizeScope* to all fragments. Compensating a scope in a loop requires compensating every instance of the scope that ran for one instance of the parent scope requesting the compensation. The coordinator must request the corresponding compensation the correct number of times and using the correct protocol instance for each of these scope instances.

Consider compensating children of a scope s due to a 'compensateScope' or 'compensate' activity, e.g. the 'compensate' in scope A's fault handler. These activities may occur only in handlers, so the corresponding states are reachable from the 'fault handling' and 'compensating' states. From either state, a fragment of s can send *defaultCompensation* or *compensateChild* to request default compensation or compensation of a child scope s' , respectively. The coordinator then triggers the necessary compensation, which it does for the default compensation case in the order determined from navigating the DCO. Once the compensation is completed, the coordinator sends back *defaultCompensationCompleted* or *childCompensationHandlerCompleted*, respectively.

Consider the transitions and states enabling a scope such as scope B to compensate an unsplit scope, such as C, nested in B and present in the relationship tree. The work is reflected in the four 'compensating non-split child' states: the protocol of B for the fragment containing C enters this state once the coordinator sends it *compensateNonSplitChild*. Once the fragment locally completes this compensation, it sends *childCompensationHandlerCompleted*. If C has not completed, the engine must treat this request as a no-op and immediately send back the completion notification. The unsplit child scopes can also be compensated from the 'terminating' and 'fault-end' states. The coordinator, in these states, is performing compensation of C as part of terminating the children of B and does so through B's protocol because there is no protocol for unsplit scopes.

Some transitions to 'aborted' not in the figure for simplicity address faults in the fault handler while compensation is running: *faultAndExit* from 'compensating non-split child(2)' and *faultAndExit* and *faultedInHandler* from the three compensation states reachable from 'fault handling'.

This leaves the coordinator behavior for compensation due to termination and rethrow of a fault, e.g. scope A faults while B is running, which uses the protocols of

multiple scopes. Recall that upon receiving a notification of a fault from a scope s_m , the coordinator continues from a potentially different scope s_f . BPEL behavior when a fault is thrown: default termination of running activities. If the scope has no handler, then do default compensation of completed scopes followed by rethrowing the fault to the parent scope. To use less messages and simplify the runtime, we drive this behavior from s_f : stop all activities in s_f then perform compensation in the order necessary by calculating what it would have been had we, at each scope between s_m and s_f , actually caught and rethrown the fault.

Consider the time the coordinator has notified the fragments of s_f of the fault, causing all active activities in s_f to be aborted by throwing the fault locally in each fragment. Due to our restrictions on the engine's ability to trigger compensation on its own, no compensation is done during this for *split* scopes in s_f . Then the coordinator drives the following behavior:

1. Starting with $s = s_m$:
 - (a) 'Compensation due to Termination' of activities in s : Perform default compensation on every split scope s_t in s , innermost first, if s_t is not in a handler of s , is not completed, and has at least one immediate child scope that is completed and in the DCO of s_t .
 - (b) If s has no handler for the fault, perform default compensation on s and if $s \neq \text{process}$ continue with 'Compensation due to Rethrow'. Otherwise skip steps 2 and 3.
2. Perform 'Compensation due to Rethrow' of the fault: Going up the relationship tree along child–parent edges from s_m to s_f , for each encountered scope node $s_p \notin \{s_m, s_f\}$:
 - (a) If s_p 's *in_fault_handler* Boolean is true, skip s_p .
 - (b) Otherwise, perform step 1 for $s = s_p$ but skip any scopes already touched by previous iterations.
 Perform 'Compensation due to Termination' of s_f :
 Perform step 2 for $s = s_f$ but skip traversing down any
3. subtrees already touched in steps 1 or.

Finally, continue as in the fault handling subset: if s_f has a matching fault handler, send *runHandler* to its fragments to run the fault handler; otherwise, send *faultAndExit* to all fragments of the process. Section 6 will explain the protocol messages used to coordinate the fragments in Fig. 1 once the fault is thrown by the 'throw' activity.

Race conditions exist when a state has transitions whose messages can be sent by either coordinator or participant. The resolution rules are: a fault related message wins over a compensation message; in races between fault related messages, those from the coordinator win over those from a participant; races between compensation messages result in queuing the messages.

5. System design and implementation

The fragmentation system [10] consists of design time and runtime subsystems. The design time component is a

BPEL editor that enables a designer to define a partition backed by a transformation module that can generate the resulting fragments. The fragments can then be deployed to the runtime subsystem for execution. The runtime subsystem consists of extended BPEL and WS-Coordination runtimes.

The design time system enables users to graphically define a partition and trigger fragment generation. An option to define participants is enabled for any process open in the editor. Once a participant is defined and named, the user can right click on any activity in the process and assign it to one of the defined participants. Once all activities have been associated with a participant, the new capability of splitting the process becomes enabled. Upon selecting that the process be split, the system will generate the processes for each fragment, the WSDL files, and the wiring definition. Since the generated processes are themselves BPEL processes, they can also be opened in the editor, enabling the designer to inspect the results in a familiar notation. The user can then use these to deploy the process to any set of BPEL runtimes if no coordination is required or to fragmentation enabled runtimes otherwise. The user can deploy the fragments directly from the editor by choosing the deploy option. This prompts the user for the necessary endpoints and deployment locations, generates the deployment artifacts required by the target runtime system(s) and transfers them to the deployment location of the corresponding BPEL engines.

A prototype, detailed in [29], was created by extending the Eclipse BPEL editor (<http://www.eclipse.org/bpel>). The prototype deploys the fragments to the ActiveBPEL runtime, extended as described next to handle the fragmentation. The prototypes currently supports BPEL-D, a version of BPEL that uses explicit data links for data dependencies.

Having described the design time subsystem, we move on to the runtime subsystem, detailed in [24] along with the WSDL definitions of the protocol services. To support the protocols, the BPEL engine(s) used must be extended to recognize the new language extensions and support the participant side of the protocols, which affect and are affected by the engine's BPEL navigation. This includes a starter service, a loop participant service, and a scope participant service. First, consider the participant side. Aiming to avoid new middleware where possible, we use a modular and minimally intrusive mechanism [11] for extending the behavior of existing BPEL engines. The mechanism uses an engine-agnostic event model for BPEL lifecycle along with a 'Generic Controller' with pluggable 'Custom Controllers' for new extensions. To support a particular BPEL engine a developer provides a module that maps between the generic event model and the native navigation model and capabilities of the engine. We extend the ActiveBPEL engine in this way as described in [11]. Then, supporting a new BPEL extension consists of providing a new Custom Controller for that extension. The mechanism supports three types of events used for the interactions between the ending and a controller: (1) 'Notification Events' are sent from the engine, causing the custom controller to react by performing a custom action (such as send a WS-Coordination message); (2)

'Incoming Events' are sent from the controller into the engine, enabling it to react to external events while processing (such as receiving a fault from another fragment); (3) 'Blocking with Callback' events, or BC events enable the engine to be sent an event and block the path of the process that produced this event. A particular Incoming Event is related with each BC event, unblocking the path and determining how the engine must continue on that path. Using this mechanism, we can add the control needed on top of a BPEL process instance based on interactions with the coordinator.

In order to support the participant side of the coordination protocols, a Split Activity Custom Controller (Fig. 10, left) is created as shown in Fig. 10. This new controller implements the participant services, the outbound message invocations they send, and the required behavioral logic. This controller interacts with the Generic Controller by receiving and sending events over JMS using ActiveMQ to control and react to the behavior of the engine as needed as well as to receive and send protocol messages using SOAP to and from the coordinator. The casual relationships between the events of the event model and the coordination messages are provided in [10]. Ref. [11] illustrates controlling the engine to execute a loop fragment.

Next, consider the coordinator. At deployment time, the coordinator endpoint is bound with the controllers and the coordinator is provided with the scope relationship tree, default compensation order graphs, and starter service addresses. The coordinator implementation extends the WS-Coordination system in [30] by plugging in the new protocols (Fig. 10, right). This consists of providing: (1) the registration handler that provides protocol registration, including handling the extensions in the registration message and using them to create a coordinator side instance identifier, seeing whether an instance of the activity already exists, creating the reference properties needed for the coordinator address sent back in the registration response message, (2) the coordinator services for incoming protocol messages and the outgoing invocation for the outbound ones, and (3) the corresponding behavioral logic such as searching for fault handlers, grouping fragments of the same instance together based on keys generated using input from the registration handler, calculating compensation order. The coordinator stores state in stable storage using MySQL database tables, initialized using the scope and loop relationship tree. The table data is used by the handler and the

coordinator logic. Upon receiving a registration message, the handler checks whether it already has a previous fragment registered for that same activity. If not, it creates a new record for that instance of the split activity. Otherwise, it adds the fragment to the table for that instance, and then creates and sends the address for the coordinator instance using the same reference properties as for other fragments of this split activity. As the messages for the protocol of that activity instance are exchanged, the tables get updated accordingly.

The runtime subsystem implementation currently extends the ActiveBPEL runtime and supports the protocols except default compensation. It requires fragments of the same process instance to interact with the same coordinator. A set of processes was created to test that the execution semantics of the unsplit process model is preserved using the implementation. Details on this validation are in [24]. The tested processes contain split loops and/or scopes, including nesting, and different variations on split fault and compensation handling.

A screen grab of running a split process with a split scope (scope_one) in which a loop (uniqueName) is nested is shown in Fig. 11. This is using the built-in monitoring capability of the ActiveBPEL engine. The 'throw' activity in the left fragment of the loop occurred after 'assign' in the right had completed. However, both fragments of the loop have an 'x' showing that the fault is registered at both fragments. Also, see how the 'throw' in the left fragment caused the fault handler in the right fragment of scope_one to execute. As desired, the fault handler ran even though no fault was encountered locally in the right fragment and navigation continued normally from scope_one even though a fault handler did not run locally in the left fragment.

6. Illustrating the coordination

Having explained the approach and the implementation, we now walk through an example of the approach using the example process in Fig. 1. A company wants to split apart its hiring process so that different parts can be handled by the departments responsible for them (potentially an external third-party), in this case HR (P1, left) and Payroll (P2, right). Some activities (to the left of the thick line) are determined to be owned by HR while those

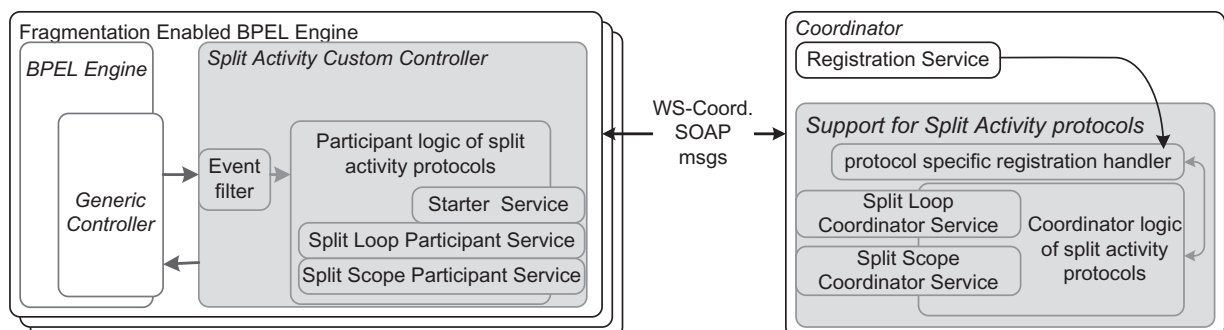


Fig. 10. Architecture of the runtime system.

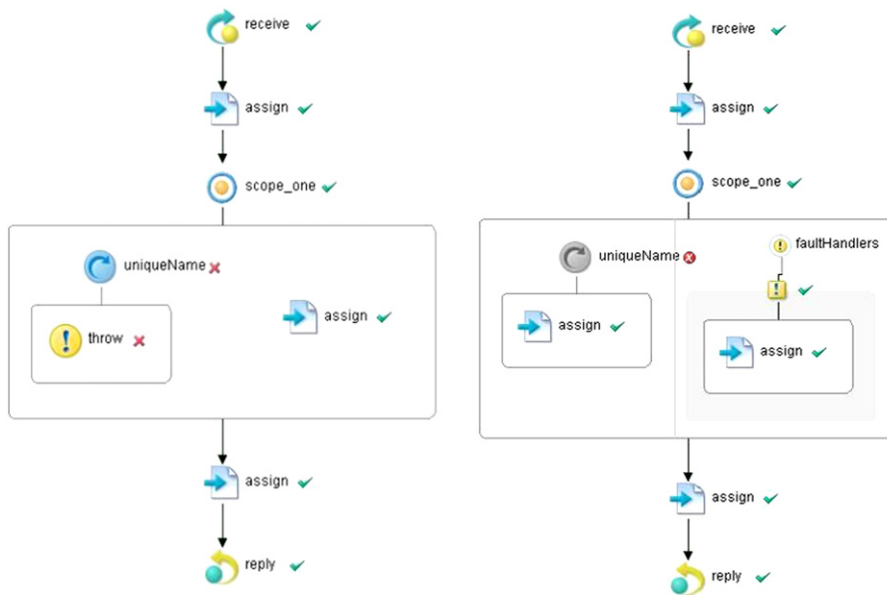


Fig. 11. Screen grabs of the ActiveBPEL monitor upon running fragments of a split loop in a split scope [24].

to the right are owned by Payroll. The designer then defines the partition and requests that the process get split. The split scopes are, as a result, the process, scope A, scope E, and scope B. Note that the compensation handler of scope E is split. The result is the process fragments on the right in Fig. 1, WSDL files, and a wiring definition. Upon providing additional information needed for deployment (endpoints, target engines), the fragments are each deployed to a fragmentation enabled runtime and the common information is sent to the coordinator.

Now consider the time when the 'throw' activity is reached and throws a fault in P2. Assume that at the time scope B is still active and C and D have completed. The fault is in the split scope A and P2 has a matching fault handler for it. We illustrate what happens next in Fig. 12, and describe in detail in the rest of this section. In the figure, time flows from top to bottom and messages surrounded by square brackets such as the two 'compensate' messages from the coordinator, can occur in any order (i.e. they are parallel). The left side brackets show what each set of messages is for, while the right side brackets show which scope protocol the messages are part of. The rest of this explanation is organized according to the headings in the left side brackets.

Fault is thrown: When the fault is thrown by the 'throw' activity in P2, P2 sends *faultedWithHandler* to the coordinator. The coordinator receives this message and determines from the relationship tree that it needs to notify the other fragment of scope A about the fault. It sends *fault* to P1. This suspends forward navigation in both fragments of scope A and recovery can now begin.

Terminate activities in scope A: The next step is to terminate the activities in scope A. Since we assume B is still active at this time, then the termination of scope B must occur next. Terminating B consists of running default compensation on its completed children. According to the DCO,

the order is D then C. Thus, the coordinator triggers compensation of scope D by sending *compensateNonSplitChild(scopeD)* to P2. P2 performs compensation and sends *childCompensationHandlerCompleted scopeD*). This is followed by similar behavior and messages for scope C in P1. This concludes the termination of B and therefore of A. These messages go through scope A's protocol because C and D are not split.

Run the fault handler and complete scope A: The next step is to run the fault handler of A. The coordinator sends *runHandler* to P2's protocol for scope A, notifying it that it may now proceed with fault handling. The handler consists of a 'compensate' activity which causes default compensation of A's nested scopes. Since A is a split scope, P2 must request this from the coordinator and sends *defaultCompensation* to the coordinator. The coordinator then refers to the DCO of A and determines that E must be compensated, sending *compensate* to both fragments of E. Note that E is a split scope itself so this message goes to the scope protocol instance of scope E, which up to this point would have been in the 'completed' state. This synchronizes the beginning of the split compensation handler in E and both fragments start. Upon completion, they each notify the coordinator by sending *childHandlerCompleting(scopeE)*. Once the coordinator receives this from both fragments, it allows them to complete by sending *finalizeScope* to each. Now, the coordinator is done with scope E and can notify scope A that its request for default compensation has been carried out. The coordinator thus sends *defaultCompensationCompleted* to scope A's protocol. Since the 'compensate' activity was the only thing in scope A's fault handler, P2 can now notify the coordinator that the fault handling is complete and sends *handlerCompleted* to the coordinator. Recall that the fragment of scope A in P1 is still blocked waiting for the fault handling to take place. Now that this

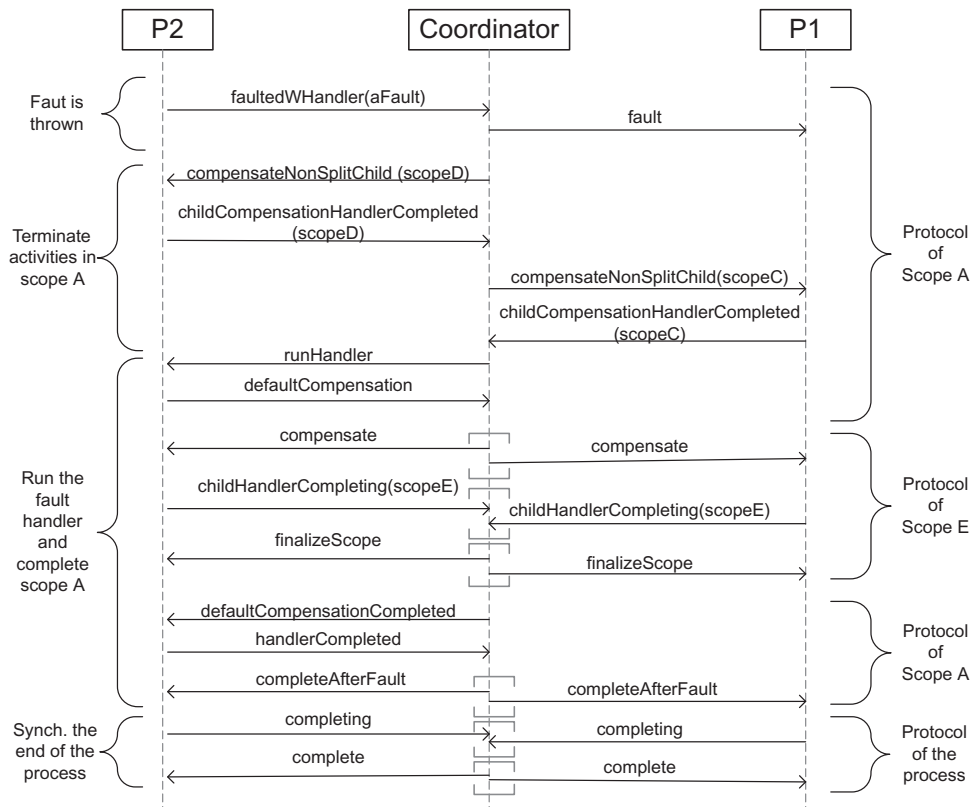


Fig. 12. Protocol messages from when the fault is thrown until the process in Fig. 1 completes.

has been done, the coordinator sends *completeAfterFault* to both fragments of scope A. This concludes scope A's protocol instance.

Synchronize the end of the process: The coordinator still has to synchronize the end of the process itself. Once each fragment has completed, in this case by executing the reply activity at the bottom, it sends *completing* to the coordinator. The coordinator waits to receive this message from both fragments and then sends *complete*. This concludes the execution of the split process.

This example illustrated how the protocol is used to execute split processes, highlighting paths of fault handling, compensation due to termination including unsplit scope, default compensation handling, and split compensation handling.

7. Related work

WS-Coordination has been used with BPEL to coordinate nested scopes in BPEL1.1 (of which sub-processes [9] are a special case) or provide transaction capabilities [27] to all interaction activities in a scope or related to a partnerLink. Ref. [25] compares WS-BusinessActivity (WS-BA) to the parent-child scope relationships in a BPEL process. Ref. [16] extends WS-BA to handle scopes spanning multiple processes and their interaction with local scopes. SELF-SERV [2] uses coordination between services in the same composition to enable rapid service composition.

There is a large body of work on distributed workflows. In [3], a publish/subscribe event mechanism wires together processes. CrossFlow [8] focuses on inter-organizational workflow. Several approaches start with one workflow and then split it apart: using state-charts [20] deriving individual workflows for each organization using Petri Nets [28], mapping from a conversation-based language (WS-CDL [18]) or from a state-machine based language [7] to BPEL. Refs. [31,6] split abstract process models, recreating dependencies using exchanged messages and [6] adds synchronization patterns for loops, multiple instances and a discriminator but no scopes. Ref. [1] provides a high level approach for splitting BPEL resulting in BPEL fragments.

Other work has distributed the execution of one workflow for runtime optimization, such as OSIRIS [26] or the tuplespace approach in [17]. Ref. [4] uses program analysis and node reordering to maximize the throughput by breaking down a BPEL process into several BPEL processes. Ref. [21] shows a model to analyze the effects of local and remote check-pointing for fault handling and envisions embedding it in agent based workflow systems for run-time fault tolerance technique selection. Aspect Oriented Programming (AOP) has been used to inject behavior into BPEL processes [5]. It may be used to add the participant side of the protocol to AOP-enabled BPEL engines. The above approaches have partial to no support for splitting compensation and fault handling. Some have different goals. The goal misalignment is due to the use of

non-BPEL meta-models (UML Activity diagrams, state charts, custom abstractions), requiring of new execution runtimes (coordination spaces, SELF-SERV, Osiris), and/or lack of transparency because runtime artifacts (tuples) are in different meta-models than build-time artifacts (process model).

8. Discussion and future work

The end-to-end approach we propose consists of splitting data and explicit control dependencies are fragmented according to [12,13] and splitting implicit control dependencies for loops and scopes described in this paper. It can handle nearly arbitrary partitioning of a BPEL process, which is quite powerful. The resulting complexity is gradual, depending on the partition. In the best case of splitting a process with our end-to-end approach, coordination is not needed, no new messages need to be exchanged and no new activities are added in the fragments. As the dependencies being split become more complex, so does the resulting fragmentation solution.

Our approach starts with creating one process containing the designated activities for each participant. The next level of complexity is introduced if split control or data dependencies are split. In this case, we add message exchanges to the processes of the participants to transmit any split control and data dependencies. The final level of complexity is introduced if fault handling, recovery, and looping behavior are split. This is handled by adding the coordination protocols and BPEL extensions that have been the focus of this paper. For example, a process consisting of two parallel paths that share no data or control can be simply split into two processes. Each process would contain one of the paths. If these paths have a control link synchronizing them, then a message exchange is added using patterns of BPEL activities in each process in order to recreate the dependency defined by the synchronizing link. If data dependencies caused by shared variables with multiple writers are split, then patterns of BPEL activities are again used. In this case [12], the complexity of the patterns of added activities and exchanged messages could increase substantially. How much so depends on how intertwined the data dependencies are and how good the data analysis algorithm used is. The final level of complexity comes if the designer needs to split loops and scopes. The process fragments themselves are not heavily impacted in this case, but more work is done in underlying protocols that use coordination to drive some of the advanced implicit control semantics of BPEL. Additionally, extensions are required for the BPEL engines and a coordinator must be used.

With respect to the runtime overhead, we consider the number of messages exchanged to handle a split loop or scope. This is expected to be on the low end of what the protocol covers: scopes usually execute successfully, in which case the number of protocol messages exchanged is four messages for each fragment of each split scope. For a split scope that encounters and handles a fault with no compensation, that number goes up to six protocol messages for each fragment of each split scope. The worst case

is expected to be quite rare: for all the states and messages in the protocol to be applicable to a scope, it would need to be a split scope having a compensation handler, containing both split and unsplit scopes that have compensation handlers, and having both split and unsplit scopes in its own compensation handler. It would be very interesting to analyze BPEL processes that are running in production to determine what advanced scope features are commonly used in practice and perform a study on what of these people would need to fragment.

We have designed the algorithms and protocols for split loops and scopes to lower overhead where possible. However, we consciously traded off some speed and lowering complexity in the middleware for transparency, interoperability, and lowering complexity in the artifacts the business user interacts with (i.e.: the process definition). For split loops and scopes, the business user sees fragments that are BPEL processes where annotations identify any split activities and potentially the addition of (identifiable) patterns of receiving and sending BPEL activities for other split dependencies. He does not see any of the machinery that runs the protocols and thus is able to easily identify how and where the business logic of the unsplit process is used in the fragments. The behavioral logic of the unsplit process, and its counterpart in the fragments, is not obfuscated by translation or layers of abstraction. Our motivation for this trade off is that the modeling and the execution of a business process are often done over an extended period of time. BPEL processes, especially those that require the quality of service provided by transactional scopes, are usually long running (days, months, or even years) and carefully modeled. We therefore expect that for partitioning processes where fragmentation of advanced recovery behavior is needed, the overhead caused by additional messages or more complexity in the algorithm that generates the fragments is not the cause of significant delay in comparison to the time it takes for a process to execute or the modelers to design the process. For processes that are short running, agile, and still require splitting complex transactions and recovery mechanisms, the trade-offs may be different thus making a different approach more suitable. In our end-to-end approach, splitting implicit data dependencies caused by shared variables is where we found the most potential for complexity in the resulting processes.

We considered several alternatives for split loops and scopes. One was to use patterns of BPEL constructs that exchange messages to synchronize the split behavior instead of using coordination. This would work for loops fragmented amongst few participants but becomes intractable for more complex cases. Another alternative is to use a new BPEL process that acts as a lightweight coordinator and mediates all the interactions. However, that did not fit our goals for several reasons: first, we want a first class separation of the business logic for each participant from the underlying BPEL execution semantics while still promoting reusability and transparency; second, it would always require a coordinator even for simple cases; third, the participants would not own their subset of the business logic. At the far end of the

spectrum, one could move to a fully distributed BPEL engine. We believe an interesting extension to this work is to explore hybrid approaches whereby a cost function could be used to recommend a partitioning strategy for a user based on the characteristics of the process, the partition, and the user's fragmentation drivers. This would require studying where the tipping points are, determining the requirements one has for needing new middleware, and whether it would be possible to create a system that can provide a seamless transition along the different points on the middleware-requirements curve based on different user needs.

Having presented and discussed the mechanism itself, we step back to how it would be used. For each partition, we envision that each participant would own and manage his or her fragment. We use the same language (BPEL) for the unsplit process as for the fragments and strive to maintain the ability to easily separate the behavior that came from the unsplit process from any additions caused by the fragmentation. By doing so, we set the stage for two areas of future research: (a) fragmentation evolution and autonomy and (2) versioning of the unsplit process and propagating changes to its partitions (and thus to the fragments of each partition). Fragmentation evolution and autonomy refers to the ability for a partner to update, modify or change his fragment while still complying with the unsplit process. BPEL abstract processes could be leveraged for this. Versioning of the unsplit process moves in the other direction. It refers to the ability of the process owner (or owners) to make changes to the unsplit process itself and have those changes reflected in some or all of the existing partitions.

Besides the long-term research directions above, near-term work includes expanding the coverage of the prototype, relaxing the restrictions on the unsplit process models and adding optimizations to the splitting algorithm by grouping of dependencies.

9. Conclusion

We have provided an automatic and operational semantics-preserving decomposition of recovery, roll-back, and looping capabilities in fragmented business processes. The approach is modular, extending existing standards and runtimes; the implementation builds on an existing BPEL engine and WS-Coordination system. The paper complements our prior work on process decomposition to result in a decomposition solution, extensively detailed in [10], for comprehensive business process functionality in a transparent and interoperable manner. The approach can also be used to enable adding fault and compensation handling scopes that span multiple *existing* processes.

Future research includes fragment evolution, autonomy and reuse, and a study of how, when and why one moves along the middleware-requirements spectrum of fragmentation solutions: from plain BPEL engines to modularly adding extensions to fully distributed process execution platforms.

Acknowledgement

Michael Paluszek's for his implementation of the runtime component and Javier Vazquez Fernandez for his implementation of the fragmentation editor.

This paper is an extended version of the paper of the same title at *BPM2010*, both based on [10].

References

- [1] L. Baresi, Towards distributed BPEL orchestrations, in: Proceedings of SeTra Workshops, EASST, 2006.
- [2] B. Benatallah, M. Dumas, Z. Maamar, Definition and execution of composite web services: the SELF-SERV project, *IEEE Data Engineering Bulletin* 25 (4) (2002).
- [3] F. Casati, A. Disenza, Supporting workflow cooperation within and across organizations, in: Proceedings of SAC, ACM, 2000.
- [4] G. Chafle, S. Chandra, V. Mann, M. Gowri Nanda, Decentralized orchestration of composite web services, in: *WWW Alternate Track*, ACM, 2004.
- [5] A. Charfi, M. Mezini, AO4BPEL: an aspect-oriented extension to BPEL, *World Wide Web Journal* 10 (3) (2007).
- [6] W. Fdhila, C. Godart, Toward synchronization between decentralized orchestrations of composite web services, in: *Proceedings of CollaborateCom*, 2009.
- [7] X. Fu, T. Bultan, J. Su, A top-down approach to modeling global behaviors of Web services, in: *Proceedings of REOS Workshop*, IEEE, 2003.
- [8] P. Grefen, Y. Hoffner, CrossFlow—cross-organizational workflow support for virtual organizations, in: *Proceedings of RIDE Workshop*, Washington, DC, IEEE, 1999.
- [9] IBM and SAP, WS-BPEL Extension for Sub-processes BPEL-SPE. <<http://www-128.ibm.com/developerworks/library/specification/ws-bpelsubproc/>>, 2005.
- [10] R. Khalaf, Supporting Business Process Fragmentation While Maintaining Operational Semantics: A BPEL Perspective, Ph.D. Thesis, University of Stuttgart, 2008. ISBN 978-3-86624-344-6, dissertation.de, <<http://elib.uni-stuttgart.de/opus/volltexte/2008/3514/>>.
- [11] R. Khalaf, D. Karastoyanova, F. Leymann, Pluggable framework for enabling the execution of extended BPEL behavior, in: *WESOA Workshop*, ICSOC, Lecture Notes in Computer Science, Springer, Vienna, Austria, 2007.
- [12] R. Khalaf, O. Kopp, F. Leymann, Maintaining data dependencies across BPEL process fragments, *International Journal of Cooperative Information Systems* 17 (3) (2008).
- [13] R. Khalaf, F. Leymann, Role-based decomposition of business processes using BPEL, in: *ICWS, Industry Track*, Chicago, USA, 2006, pp. 770, 780.
- [14] R. Khalaf, D. Roller, F. Leymann, Revisiting the behavior of fault and compensation handlers in WS-BPEL, in: *Proceedings of CoopIS, OTM, Vilamoura, Portugal*, 2009.
- [15] O. Kopp, R. Khalaf, F. Leymann, Deriving explicit data links in WS-BPEL processes, in: *Proceedings of SCC, Industry Track*, IEEE, Honolulu HI, July 2008.
- [16] O. Kopp, F. Leymann, The influence of an external transaction on a BPEL scope, in: *Proceedings of CoopIS, OTM, Lecture Notes in Computer Science*, Springer, Vilamoura, Portugal, 2009.
- [17] D. Martin, D. Wutke, F. Leymann, Synchronizing control flow in a tuplespacebased, distributed workflow management system, in: *Proceedings of ICEC*, ACM, 2008.
- [18] J. Mendling, M. Hafner, From inter-organizational workflows to process execution: generating BPEL from WS-CDL, in: *Proceedings of OTM*, Springer, 2005.
- [19] S. Moser, A. Martens, K. Görlach, W. Amme, A. Godlinski, Advanced verification of distributed WS-BPEL business processes incorporating CSSA-based data flow analysis, in: *Proceedings of SCC 2007*, IEEE, Salt Lake City, Utah, July 2007.
- [20] P. Muth, D. Wotke, J. Weissenfels, A.K. Dittrich, G. Weikum, From centralized workflow specification to distributed workflow execution, *Journal of Intelligent Information Systems* 10 (2) (1998).
- [21] J. Nichols, H. Demirkan, M. Goul, Towards a model of fault tolerance technique selection in static and dynamic agent-based inter-organizational workflow management systems, in: *Proceedings of HICSS*, IEEE, 2005.

- [22] OASIS. Web Services Business Process Execution Language (WS-BPEL) Version 2.0. <<http://docs.oasis-open.org/wsbpel/2.0/wsbpel-v2.0.html>>, 2007.
- [23] OASIS. WS-TX 1.2 OASIS Standards. <http://www.oasis-open.org/committees/tc_home.php?wg_abbrev=ws-tx#technical>, 2007.
- [24] M. Paluszek, Coordinating Distributed Loops and Fault Handling, Transactional Scopes Using WS-Coordination Protocols Layered on WS-BPEL Services, University of Stuttgart Diploma Thesis 2586, 2007.
- [25] P. Sauter, I. Melzer, A comparison of WS-BusinessActivity and BPEL4WS long-running transaction, in: *Kommunikation in Verteilten Systemen*, 2005.
- [26] C. Schuler, R. Weber, H. Schuldt, H.J. Schek, Peer-to-peer process execution with Osiris, in: *Proceedings of ICSSOC, Lecture Notes in Computer Science*, Springer, Trento, Italy, 2003.
- [27] S. Tai, R. Khalaf, T. Mikalsen, Composition of coordinated Web services, in: *Proceedings of International Conference on Middleware*, Springer, 2004.
- [28] W.M.P. van der Aalst, M. Weske, The P2P approach to interorganizational workflows, in: *Proceedings of CAiSE*, Springer, Berlin, Germany, 2001.
- [29] F.J. Vazquez Fernandez, BPEL with Explicit Data Flow: Model, Editor, and Partitioning Tool, Diploma Thesis 2616, University of Stuttgart, Faculty of Computer Science, Electrical Engineering, and Information Technology, 2007.
- [30] T. Vetter, Anpassung und implementierung verschiedener transaktionsprotokolle auf WS-Coordination, University of Stuttgart Diploma Thesis 2386, 2006.
- [31] U. Yildiz, C. Godart, Towards decentralized service orchestrations, in: *Proceedings of ACM Symposium on Applied Computing*, ACM, New York, NY, 2007.